

Marketplace Search

Current state of ms_marketplace, the agreed design for filtered search, the implementation plan, and how it works once built. Prepared 2026-10-03, before any implementation.

Summary

- Today the marketplace lists **every** public dictionary in every language. A learner of German sees French, Turkish and Lithuanian dictionaries too.
- We will add four optional filters to the two existing browse endpoints (newest, popular): **language pairs**, **tags**, and **publisher display name**.
- **The database is not the bottleneck.** With the right indexes each search is two small indexed queries. The real work is getting **tags** and **display names** into the marketplace's own table: neither is there today, and no event carries them yet.
- Three phases, ordered by how much they depend on other services: **A** languages (marketplace only), **B** tags (with ms_dictionary), **C** display names (with ms_user). Phase D updates the docs.
- **GraphQL is not used.** It would add a second API stack without making a single query faster.

1. Current state of the marketplace

ms_marketplace (port 8087, database vdbmarket) is built and secured. Phase 4 tasks P4-01 to P4-10 are done. It serves browse entirely from its own read model, which ms_dictionary keeps current through RabbitMQ events. It never calls ms_dictionary while answering a request.

Endpoints today

Endpoint	What it does
GET /marketplace/dictionaries	All listed dictionaries, newest first
GET /marketplace/dictionaries/popular	All listed dictionaries, most imported first
GET /marketplace/dictionaries/language?from=&to;=	One exact language pair, one direction only
GET /marketplace/dictionaries/publisher/{publisherId}	"More from this publisher", by JWT subject
POST /marketplace/dictionaries/{id}/import	Import into the caller's vault; publishes dictionary.imported

All reads are paged (page from 0, size 1–100, default 20) and return PageResponse with totalElements and totalPages. Only listed rows (is_listed = true) are ever returned.

Data the marketplace holds

Table	Columns that matter for search	Fed by
dictionary_stats	dictionary_id, fk_user_id (publisher), name, from_lang, to_lang (uppercase), is_listed, import_count, published_at, source_updated_at	dictionary.visibility.public / .private, dictionary.updated, dictionary.deleted, nightly dictionary.snapshot; user.deleted
dictionary_imports	dictionary_id, fk_user_id, imported_at	The import endpoint

What is missing for the search we want

Filter	Where the data lives now	In the marketplace?
Language pairs	dictionary_stats.from_lang / to_lang	Yes, but only single-pair, one-direction lookup
Tags	ms_dictionary, table dictionary_tags	No. Tag changes publish no event
Publisher name	ms_user, users.display_name (optional)	No. Listings carry only publisherId (backlog BL-04)

Other limits of the current version: the existing sort indexes are single-column and do not match the real sort order; every page runs a second `COUNT ( * )` query; and the language filter is direction-sensitive, so `DE→TR` misses `TR→DE` dictionaries.

2. Agreed decisions

Topic	Decision
Languages	The client sends exact pairs : <code>pair=EN-TR&pair=FR-DE</code> . Direction is ignored, so this returns <code>EN→TR</code> , <code>TR→EN</code> , <code>FR→DE</code> and <code>DE→FR</code> and nothing else. When the Forum view opens, the client fills in the pairs from the user's own dictionaries.
Tags	Any match : a dictionary qualifies if it has at least one of the requested tags. Tags are normalised (trimmed, lowercase), the same way <code>ms_dictionary</code> stores them.
Publisher	Matched on display name , not the Keycloak username (which can be an email). Case-insensitive substring: <code>"ann"</code> and <code>"nna"</code> both find <code>"Anna Bauer"</code> . Duplicate names are fine.
Name required	A user must accept the marketplace terms and enter a display name in the client before using the marketplace. The backend also enforces it: a listing whose publisher has no display name is not shown.
Paging	Infinite scroll. Responses carry <code>hasNext</code> instead of totals, so no count query runs.
Sorting	Unchanged: newest first, or most imported. Both accept the same filters.
Client behaviour	Language and tag chips can re-query on every change; the client cancels the previous request first. The name search runs when the user taps Search.
Name search	Not added.
GraphQL	Not used. See section 5.

3. Implementation plan

Each phase is a roadmap task (P4-11 to P4-13) and ships on its own. A needs no other service; B needs `ms_dictionary`; C needs `ms_user`.

Phase A: language pairs, paging, indexes (`ms_marketplace` only)

- Migration: add a `lang_pair` column holding the pair in alphabetical order (`DE→TR` and `TR→DE` are both `DE-TR`), and backfill existing rows.
- Every write path (publish, update, hide, nightly reconciliation) sets `lang_pair` next to the uppercase normalisation it already does.
- Indexes, all partial on `is_listed`: `(lang_pair)`, `(published_at DESC, dictionary_id)`, `(import_count DESC, published_at DESC, dictionary_id)`. The two single-column sort indexes they replace are dropped.
- A filter object and a JPA Specification builder: a condition is added only for filters the request sends.
- Pair validation: format `XX-YY`, both codes supported, two different codes, at most 10 pairs; otherwise 400.
- A new `SliceResponse` (items, page, size, hasNext) for GET and GET `/popular`. GET `/language` is removed; `/publisher/{id}` stays.
- Unit tests for pair building and the Specification; web-slice tests for every validation case.

Agent: spring-boot-development.

Phase B: tags (ms_dictionary + ms_marketplace)

- ms_dictionary: add tags to the dictionary.visibility.public, dictionary.updated and snapshot payloads (events always carry the full state).
- ms_dictionary: adding or removing a tag on a **public** dictionary also updates the dictionary's updatedAt and sends dictionary.updated. Without that bump the marketplace would discard the event as out of date, because it only accepts events newer than what it holds.
- ms_marketplace: a tags text[] column with a GIN index. Each consumer replaces the array, and the nightly reconciliation compares tags too.
- The "any tag" filter, and tags on every listing in the response.

Agent: spring-boot-architect (an event change across two services).

Phase C: publisher display names (ms_user + ms_marketplace; completes BL-04)

- ms_user: a new event, user.profile.updated {keycloakId, displayName, updatedAt, eventTimestamp}, sent after commit when the display name is set, changed or cleared.
- ms_marketplace: a new publishers table (keycloak_id, display_name, source_updated_at), the pg_trgm extension and a trigram index for substring search. The consumer ignores out-of-date events; user.deleted also removes the row.
- The name filter (at least 3 characters; % and _ in the input are escaped) and the rule that listings without a display name are hidden from every browse endpoint.
- publisherName on every listing, loaded with one batched lookup per page.

Agent: spring-boot-architect.

Phase D: documentation

- verborum.md: the API contract table and the routing-key table (two changed events, one new).
- Marketplace client guide: the pair format, the slice response, the client behaviour above, and the breaking changes below. The guide has not been sent yet, so these can go into that first message.
- The ms_marketplace, ms_dictionary and ms_user CLAUDE.md files; roadmap entries P4-11 to P4-13.

Breaking changes for the client apps

- GET /marketplace/dictionaries/language is removed; use pair=.
- Browse responses carry hasNext instead of totalElements and totalPages.
- Listings whose publisher has no display name no longer appear.

4. How it works when finished

The request

```
GET /marketplace/dictionaries?pair=EN-TR&pair=FR-DE&tag=food&tag=travel&publisher=nna&page=0&size=20
GET /marketplace/dictionaries/popular?<same filters>
GET /marketplace/dictionaries/publisher/{publisherId}
POST /marketplace/dictionaries/{dictionaryId}/import
```

Every filter is optional. Values of one filter are combined with OR, different filters with AND. No filters at all returns every listed dictionary, as today.

The response

```
{
  "items": [{
    "dictionaryId": "3f1c...", "publisherId": "a91e...", "publisherName": "Anna Bauer",
    "name": "Kitchen words", "fromLang": "TR", "toLang": "EN",
    "tags": ["food", "travel"], "importCount": 42, "publishedAt": "2026-10-01T09:00:00Z"
  }],
  "page": 0, "size": 20, "hasNext": true
}
```

What the database runs

```
SELECT ... FROM dictionary_stats d
WHERE d.is_listed
  AND d.lang_pair IN ('EN-TR', 'DE-FR')           -- both directions of both pairs
  AND d.tags && ARRAY['food', 'travel']           -- any tag, GIN index
  AND EXISTS (SELECT 1 FROM publishers p          -- name required, and the name filter
              WHERE p.keycloak_id = d.fk_user_id
                 AND p.display_name <> ''
                 AND p.display_name ILIKE '%nna%') -- trigram index
ORDER BY d.published_at DESC, d.dictionary_id
LIMIT 21 OFFSET 0;                               -- one extra row tells us hasNext

SELECT keycloak_id, display_name FROM publishers WHERE keycloak_id IN (...up to 20 ids);
```

Two small indexed queries per request: no count, no duplicate rows, no N+1 lookups.

How the data gets there

```
ms_dictionary --visibility.* / updated / deleted / snapshot (+ tags)--> dictionary_stats
ms_user       --user.profile.updated (new)-----> publishers
ms_user       --user.deleted-----> both tables
client        --GET with filters--> 2 indexed queries --> one slice back
```

Data model at the end

Table	New or changed	Purpose
dictionary_stats	+ lang_pair, + tags text[], new partial indexes	One row per listing; filters and sorts run here
publishers	New: keycloak_id, display_name, source_updated_at	Display names, kept current by ms_user's event
dictionary_imports	Unchanged	Who imported what

5. Performance and the GraphQL question

Concern	How it is handled
Count query on every page	Removed: infinite scroll only needs hasNext
Sort with filters	Partial indexes that match the sort order exactly, so Postgres stops after 21 rows
Both directions of a pair	One canonical lang_pair value, one indexed IN lookup
Tag and name filters	GIN index on the tag array; trigram index on display names
Out-of-order responses in the client	The client cancels the previous request when a filter changes
Deep scrolling	OFFSET is fine at current scale. If it ever is not, the stable dictionaryId tiebreaker allows a switch to cursor paging

Why not GraphQL. GraphQL changes how a client describes the data it wants, not how fast the data is fetched: this search costs one SQL query either way. A listing has about nine fields, so letting the client choose fields saves almost nothing. Meanwhile it would add a second API stack, N+1 risks in resolvers, harder HTTP caching, and per-field security and query-cost limits. It earns its place when many different clients build screens from data spread across several services; the API gateway can cover that later if it is ever needed.